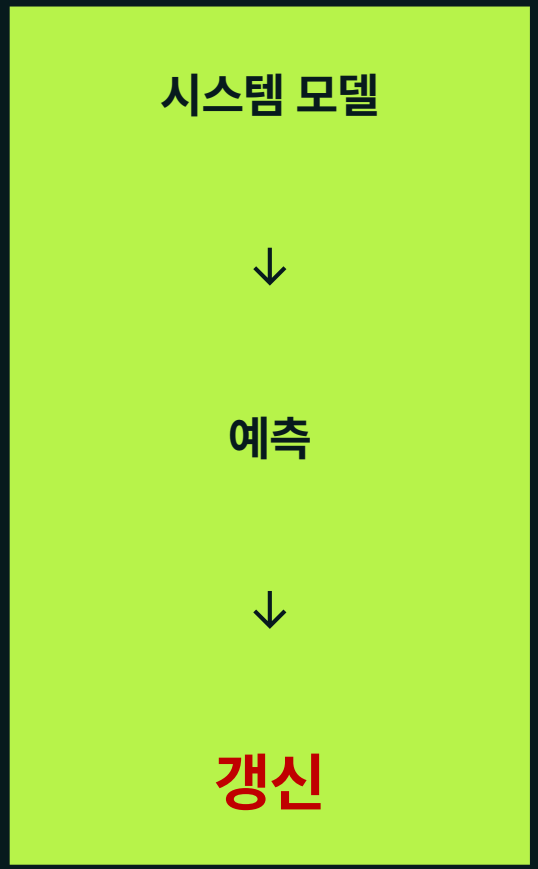
칼만 필터의 기본 모델

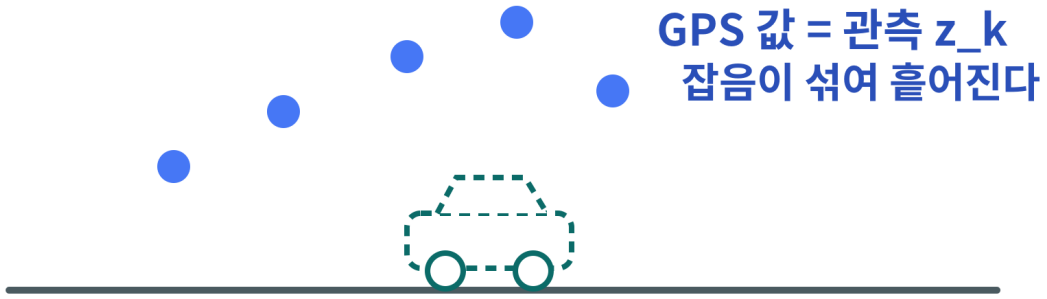
두 개의 식과 두 단계의 반복



칼만 필터는 보이지 않는 상태를 시끄러운 관측으로 추정합니다

큰 그림 — 두 식으로 문제를 적고, 두 단계를 반복해 푼다

예: 자동차 위치 추적



실제 위치 = 상태 x_k
직접 볼 수 없다

할 일: 흩어진 관측에서 실제 위치를 가장 잘 짐작하기

칼만 필터 = 두 식 + 두 단계

시스템 모델 (두 식)
상태가 어떻게 움직이고, 어떻게 측정되나

1단계 예측
규칙으로
다음 상태를 짐작

→

2단계 갱신
관측으로
짐작을 고친다

←

관측이 들어올 때마다 반복

먼저 두 식(시스템 모델)을 알고, 그다음 두 단계를 봅니다

상태식은 상태가 시간에 따라 움직이는 규칙입니다

시스템 모델 ① — 상태식(state equation)

$$x_k = A\,x_{k-1} + B\,u_{k-1} + w_{k-1}$$

$A\,x_{k-1}$

규칙대로 옮긴다

어제 상태에 규칙 A를 적용
예: 위치 + 속도 × 시간

$B\,u_{k-1}$

외부 입력

밖에서 넣는 힘
예: 가속 페달 — 금융 예제는 0

w_{k-1}

과정잡음

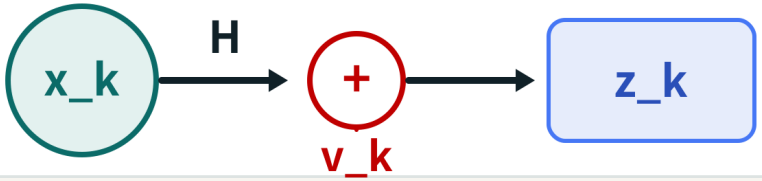
규칙으로 못 잡는 흔들림
평균 0, 크기(분산) Q

다음 상태 = 규칙대로 옮긴 값 + 외부 입력 + 잡음

관측식은 숨은 상태가 관측값이 되는 규칙입니다

시스템 모델 ② — 관측식(measurement equation)

$$z_k = H\,x_k + v_k$$



$H\,x_k$

측정 단위로 바꾼다

센서가 보는 숫자로 바꾸는 규칙
예: 위치 → GPS 화면 숫자

v_k

관측잡음

센서가 내는 오차
평균 0, 크기(분산) R

z_k

관측값

우리가 실제로 보는 숫자
칼만 필터의 입력

관측값 = 상태를 측정 단위로 바꾼 값 + 잡음

모델에 나오는 기호는 열 개뿐입니다

색은 덱 전체에서 같다 — 티일은 상태, 파랑은 관측, 앰버는 과정잡음, 빨강은 관측잡음

x_k

상태

알고 싶지만
직접 못 보는 값

z_k

관측값

센서·자료로
보는 숫자

A

상태 전이 행렬

상태를 다음 시점
으로 옮기는 규칙

H

관측 행렬

상태를 관측 숫자로
바꾸는 규칙

u_{k-1}

제어 입력

밖에서 넣는 힘
(금융 예제는 없음)

Q

과정잡음

상태가 스스로
흔들리는 크기

R

관측잡음

관측이 틀리는
크기

P

추정 불확실성

내 짐작이 얼마나
흔들리는가

K

칼만 이득

관측을 얼마나
받아들일지

$\hat{x}_k^-,\ \hat{x}_k$

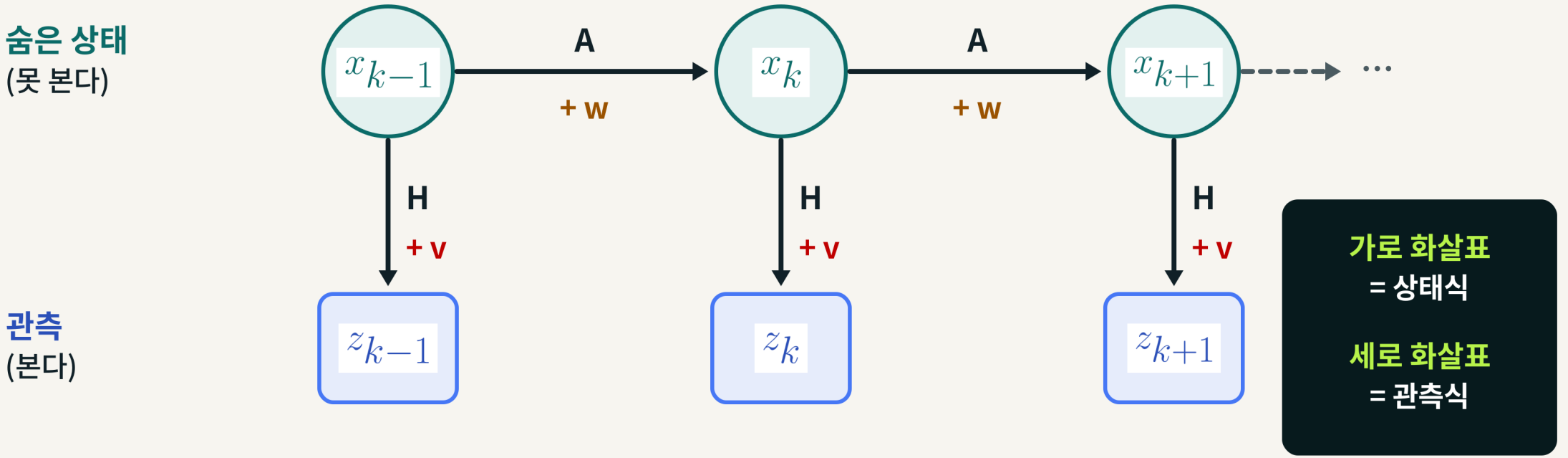
예측값 · 추정값

관측 전 짐작,
관측 후 고친 값

앞으로 나오는 모든 식은 이 열 개로만 쓰입니다

두 식을 그림으로 그리면 상태의 사슬과 관측의 줄이 됩니다

위 줄은 볼 수 없는 상태, 아래 줄은 볼 수 있는 관측



칼만 필터는 아래 줄만 보고 위 줄을 거꾸로 추정합니다

1단계 예측은 규칙대로 옮기고 불확실성을 키웁니다

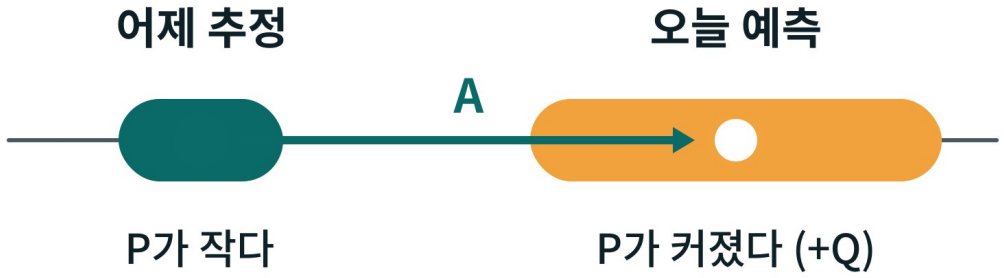
예측(predict) — 관측값이 들어오기 전에 하는 계산

$$\hat{x}_k^- = A\,\hat{x}_{k-1} + B\,u_{k-1}$$

$$P_k^- = A\,P_{k-1}A^T + Q$$

① 어제 추정값을 규칙 A로 옮긴다
② 불확실성도 옮기고 Q만큼 키운다

아직 관측값 z는 쓰지 않는다



두 식 모두 상태식에서 그대로 나옵니다

2단계 갱신은 관측으로 예측을 고칩니다

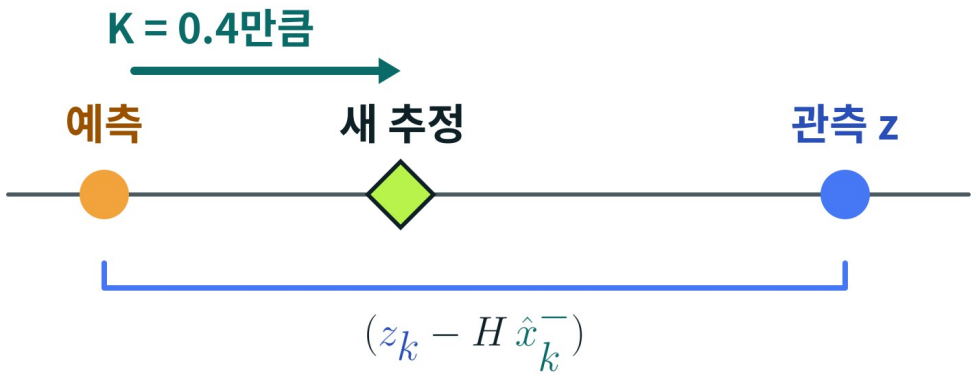
갱신(update) — 이득 K를 구하고, 혁신에 K를 곱한 만큼 움직인다

$$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1}$$

$$\hat{x}_k = \hat{x}_k^- + K_k (z_k - H\,\hat{x}_k^-)$$

$$P_k = (I - K_k H)\, P_k^-$$

③ 이득 K를 구한다 ④ 혁신 × K만큼 고친다
⑤ 불확실성 P를 줄인다



혁신(innovation) = 관측이 예측과 다른 만큼

새 추정값은 예측값에서 혁신의 K배만큼 관측 쪽으로 옮긴 값입니다

칼만 이득은 예측과 관측 사이의 비중입니다

숫자 하나짜리 모형(H = 1)에서 본 이득 — 막대의 앰버 몫이 곧 K다

$$K = \frac{P^-}{P^- + R} \quad (H = 1)$$

H = 1인 숫자 하나 모형에서
K = 전체 불확실성 중
예측이 차지하는 몫

K → 0: 예측을 믿는다
K → 1: 관측을 믿는다

예측이 불확실할 때



K = 4 ÷ (4 + 1) = 0.8 → 혁신의 80%를 받아들인다

관측이 시끄러울 때

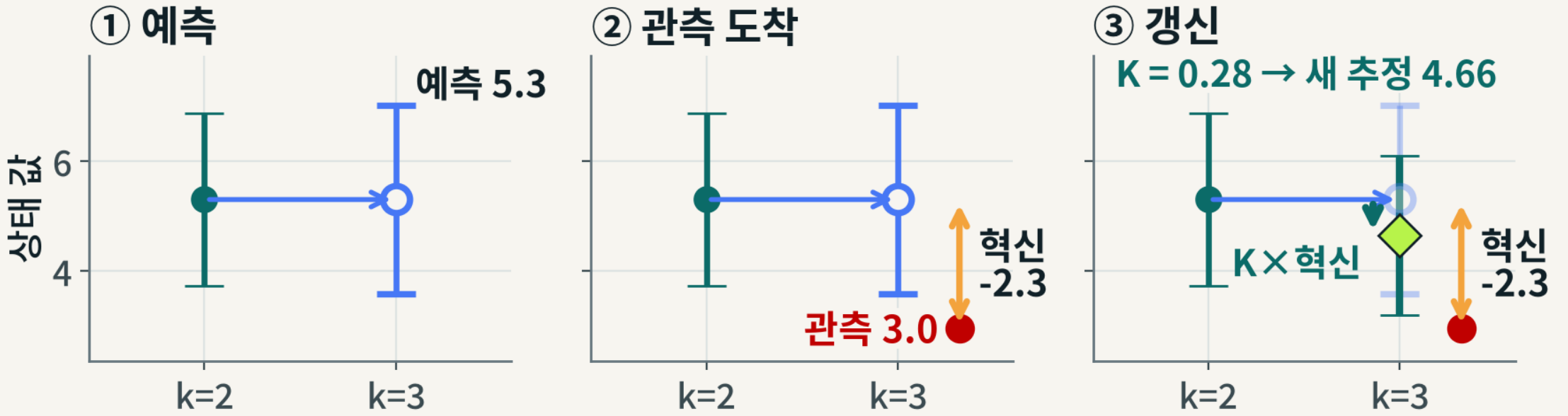


K = 1 ÷ (1 + 4) = 0.2 → 혁신의 20%를 받아들인다

이득은 고르는 값이 아니라 P와 R에서 계산되는 값입니다

한 시점에서 예측과 갱신은 이렇게 진행됩니다

k=3 한 점을 확대 — 예측 5.3, 관측 3.0, 이득 0.28

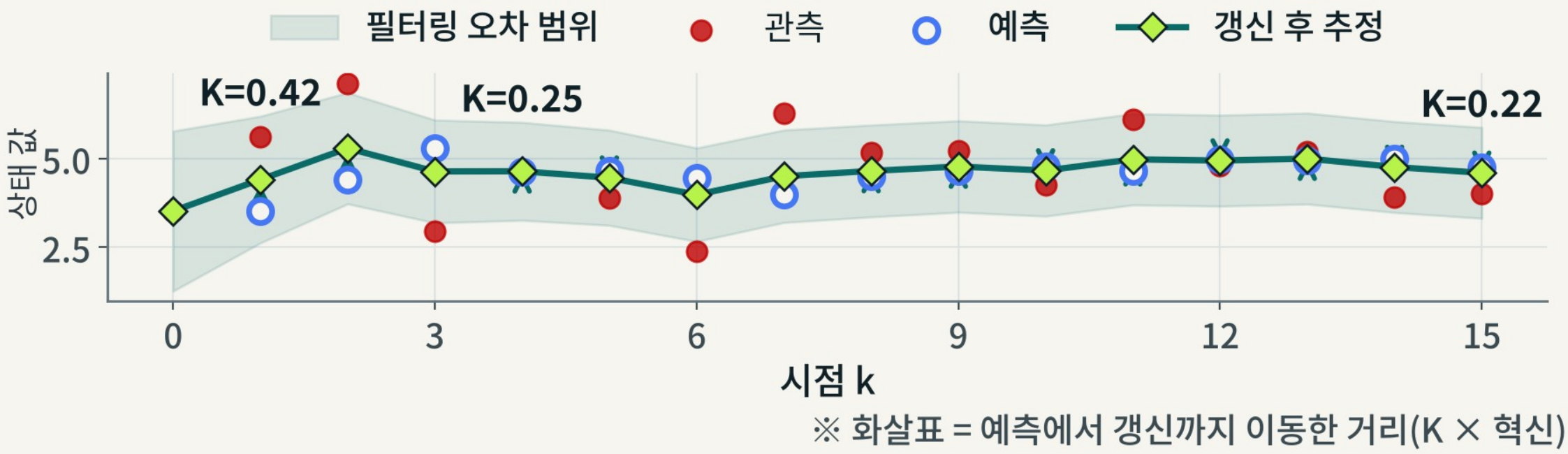


※ 세로 막대 = 95% 오차 범위(±1.96√P), 파이썬 예제와 같은 모의 자료

새 추정 4.66 = 예측 5.3 + 0.28 × 혁신(−2.3), 오차 범위도 좁아집니다

두 단계를 반복하면 추정값이 한 점씩 앞으로 그려집니다

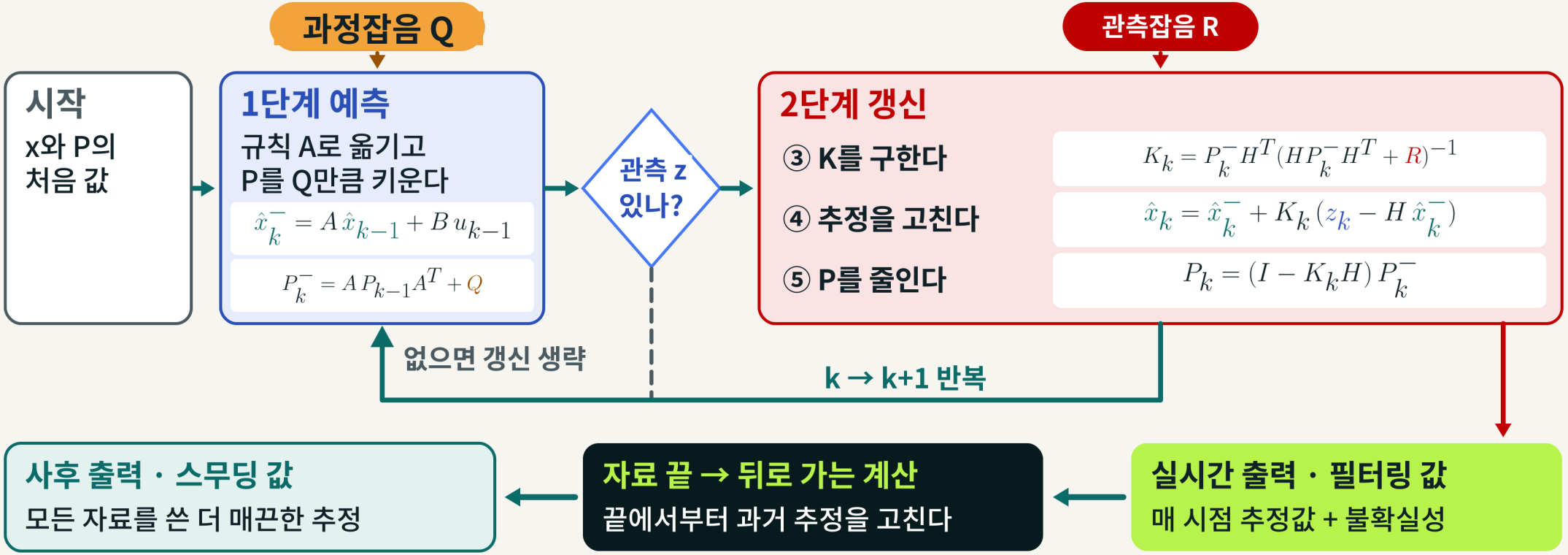
첫 16 시점 — 빈 원은 예측, 마름모는 갱신 후 추정값



처음에는 이득이 커서 크게 움직이고, 곧 일정한 크기로 자리 잡습니다

전체 흐름은 초기값에서 출발하는 하나의 반복 루프입니다

워크플로우 — 관측이 들어올 때마다 예측 두 식, 갱신 세 식을 차례로 계산한다



파이썬 코드는 다섯 식을 한 줄씩 옮긴 것입니다

numpy만 쓴다 — 오른쪽은 각 줄이 계산하는 식이다 (숫자 하나 모형, 제어 입력 없음)

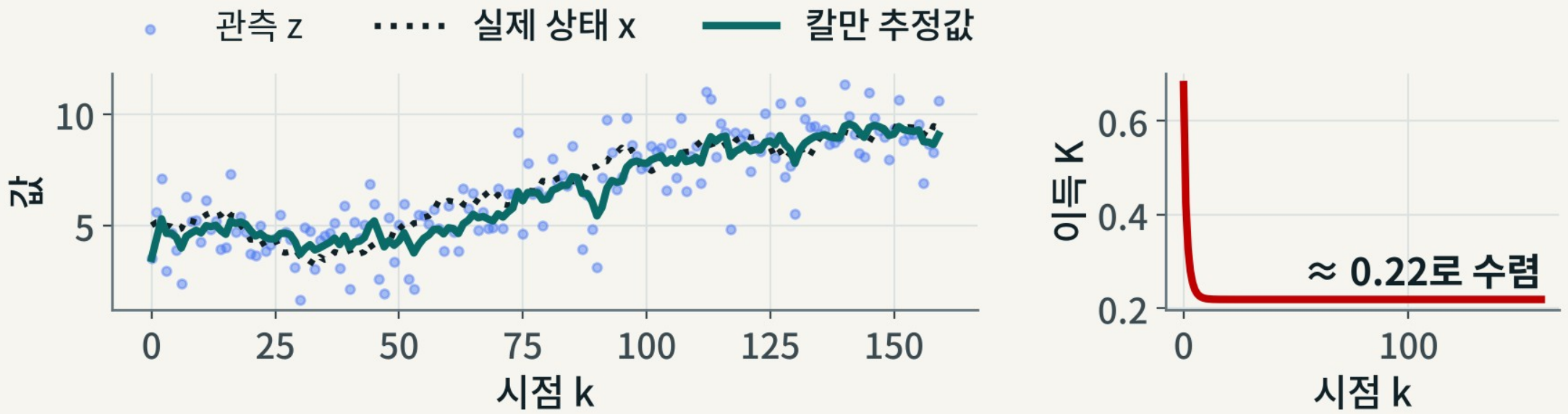
```
A, H = 1.0, 1.0
Q, R = 0.12, 1.96
x, P = z[0], 4.0
for zk in z:
    x_pred = A * x
    P_pred = A * P * A + Q
    K = P_pred*H / (H*P_pred*H + R)
    x = x_pred + K * (zk - H*x_pred)
    P = (1 - K*H) * P_pred
```

규칙	$A = 1, \quad H = 1$
잡음	$Q, \quad R$
초기값	$\hat{x}_0, \quad P_0$
①	$\hat{x}_k^- = A\,\hat{x}_{k-1} + B\,u_{k-1}$
②	$P_k^- = A\,P_{k-1}A^T + Q$
③	$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1}$
④	$\hat{x}_k = \hat{x}_k^- + K_k\,(z_k - H\,\hat{x}_k^-)$
⑤	$P_k = (I - K_k H)\,P_k^-$

코드 읽는 법 · x, P = z[0], 4.0: 첫 관측값과 4.0을 x와 P에 넣는다 · for zk in z: 관측값을 하나씩 꺼내 ①~⑤ 다섯 줄을 반복한다
x와 P는 다음 반복에서 다시 쓰고, x_pred·P_pred·K는 반복마다 새로 계산한다

실행하면 시끄러운 관측에서 실제 상태에 가까운 선을 얻습니다

위 코드를 모의 관측 160개에 돌린 결과 — 오른쪽은 시점별 이득 K

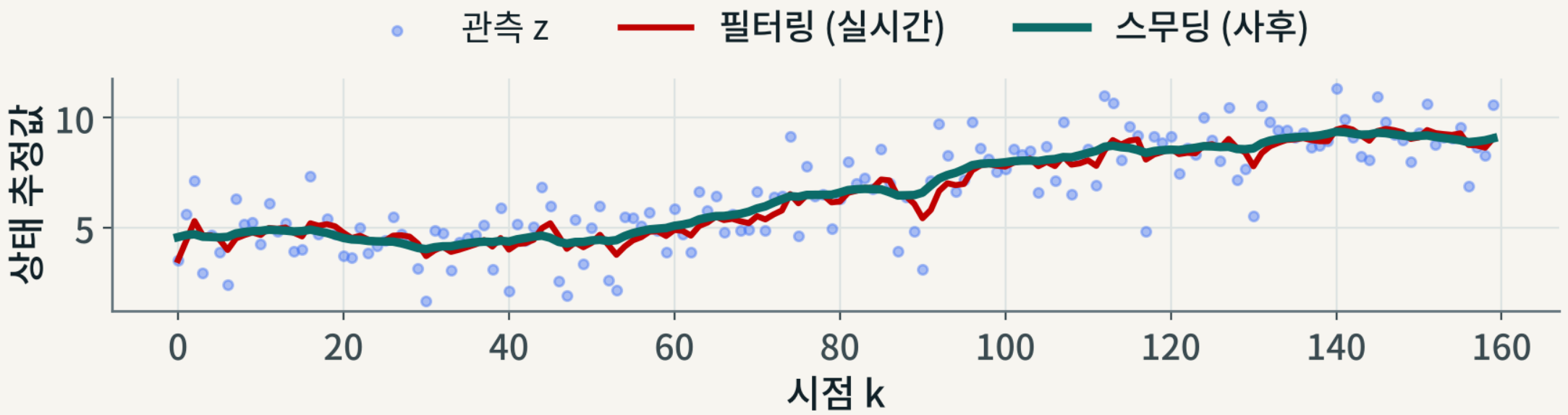


※ 모의 데이터 · Q = 0.12, R = 1.96 · 오차(RMSE) 관측 1.33 → 칼만 0.66

추정 오차가 관측 오차의 절반 아래로 줄었습니다

자료가 끝난 뒤 거꾸로 한 번 더 계산하면 스무딩이 됩니다

필터링(실시간, 앞으로 한 번)과 스무딩(사후, 앞으로 + 뒤로)



※ 모의 데이터 예시입니다

거래에 쓰는 값은 필터링 값입니다 — 스무딩 값은 나중에야 얻습니다

강의 PDF와는 같은 식을 다른 글자로 씁니다

기호 대응표 — 빨간 줄의 글자가 서로 다르니 주의한다

이 덱 (기본 모델 표기)		강의 PDF 표기
상태	x_k	x_t
상태 전이 행렬	A	T
관측 행렬	H	Z
관측값	z_k	y_t
과정잡음	Q	Q
관측잡음	R	H
혁신 (관측 - 예측)	z_k − H 예측값	v_t
시점	k	t

주의

강의 PDF의 H는 **관측잡음**,
이 덱의 H는 **관측 행렬이다**

같은 글자,
다른 뜻

식의 모양과 순서는 완전히 같습니다

두 식: 상태식과 관측식.

두 단계: 예측과 갱신.

이득 K가 둘 사이의 비중을 정한다.

다음: 이 기본식의 자리에
무엇을 넣으면 응용이 되는가